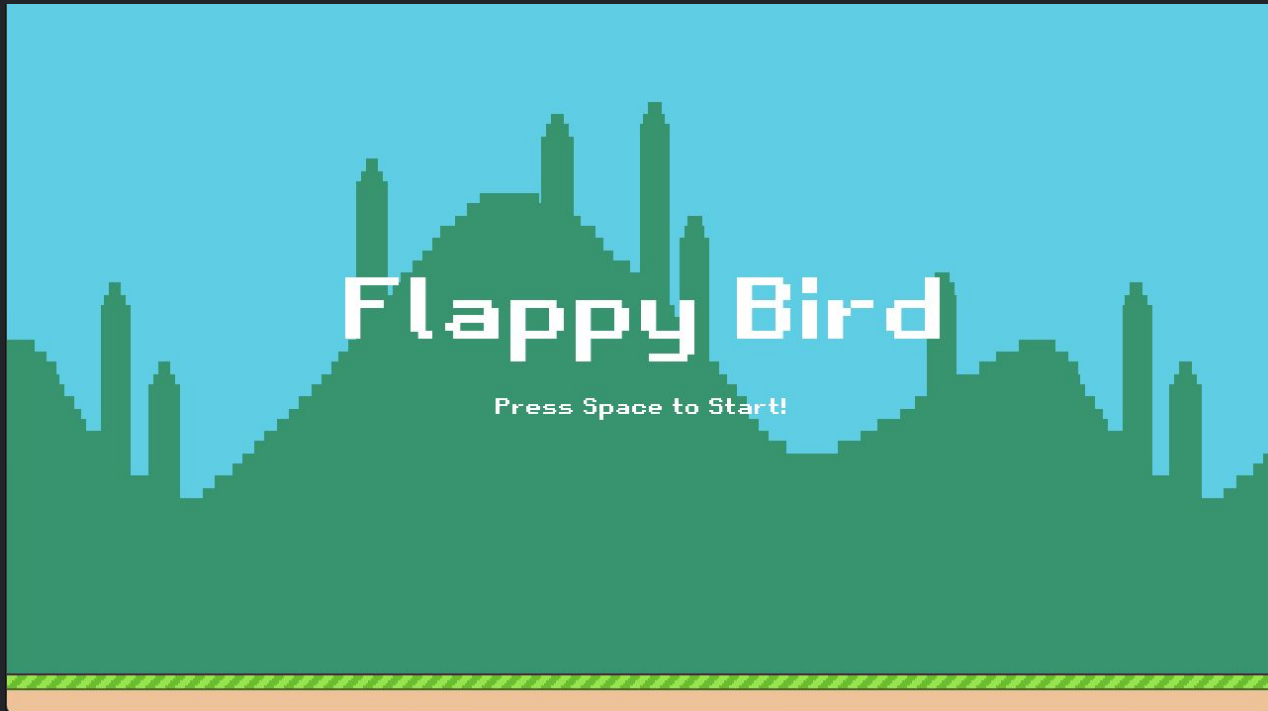


GMD2

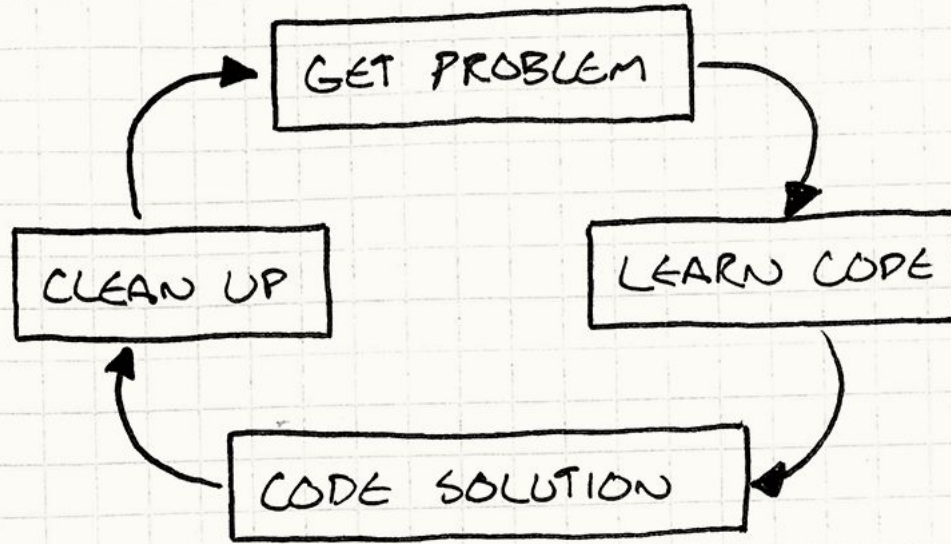
Lecture 03: Flappy Bird

Today's Goal

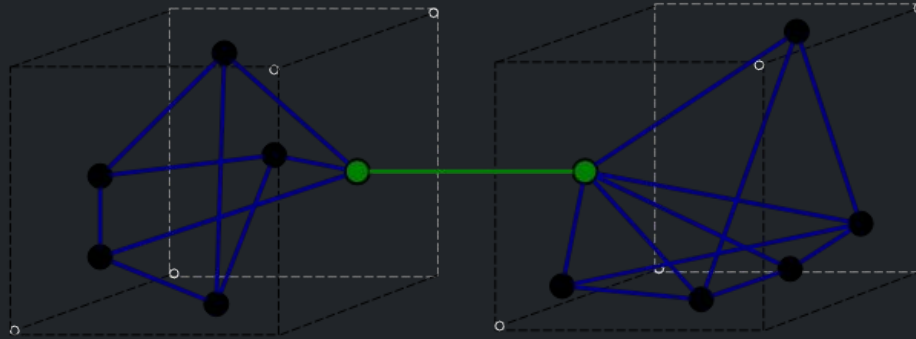


Today's Topics

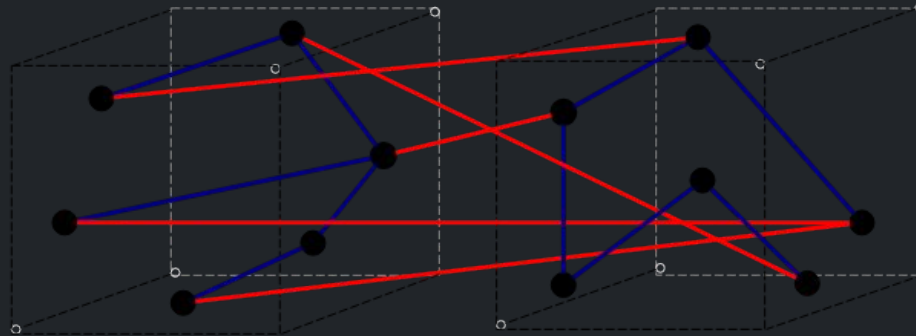
- Game Architecture
- Class Libraries
- Images
- Infinite Scrolling
- Procedural Generation
- Interfaces
- State Machines
- Singleton Pattern



Coupling & Cohesion



a) Good (loose coupling, high cohesion)



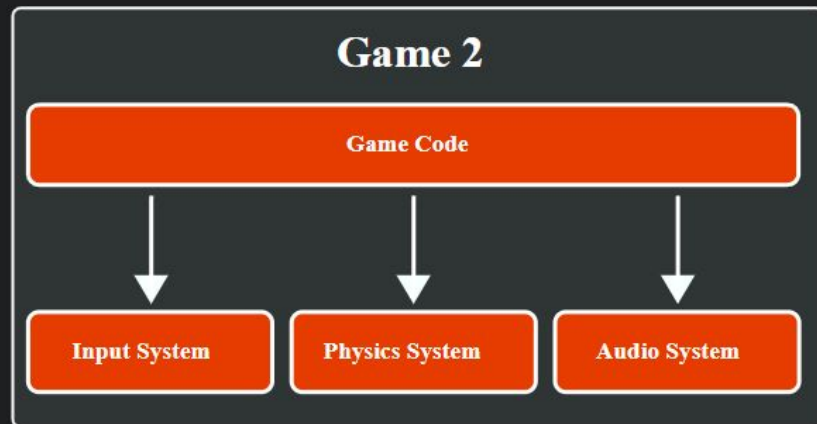
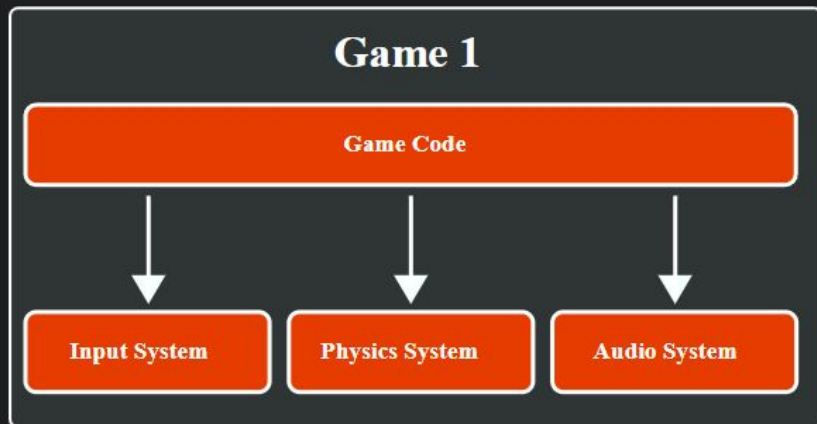
b) Bad (high coupling, low cohesion)

A detailed technical drawing of a mechanical assembly, possibly a pump or motor, rendered in blue lines on a white background. The drawing is oriented diagonally. It features various components, including a large cylindrical housing, a central shaft or rotor, and several smaller parts like bearings and seals. Dimensions are indicated with numbers and leader lines. Cross-sections are shown with hatching to indicate different materials or internal structures. The drawing is a precise engineering representation of a mechanical system.

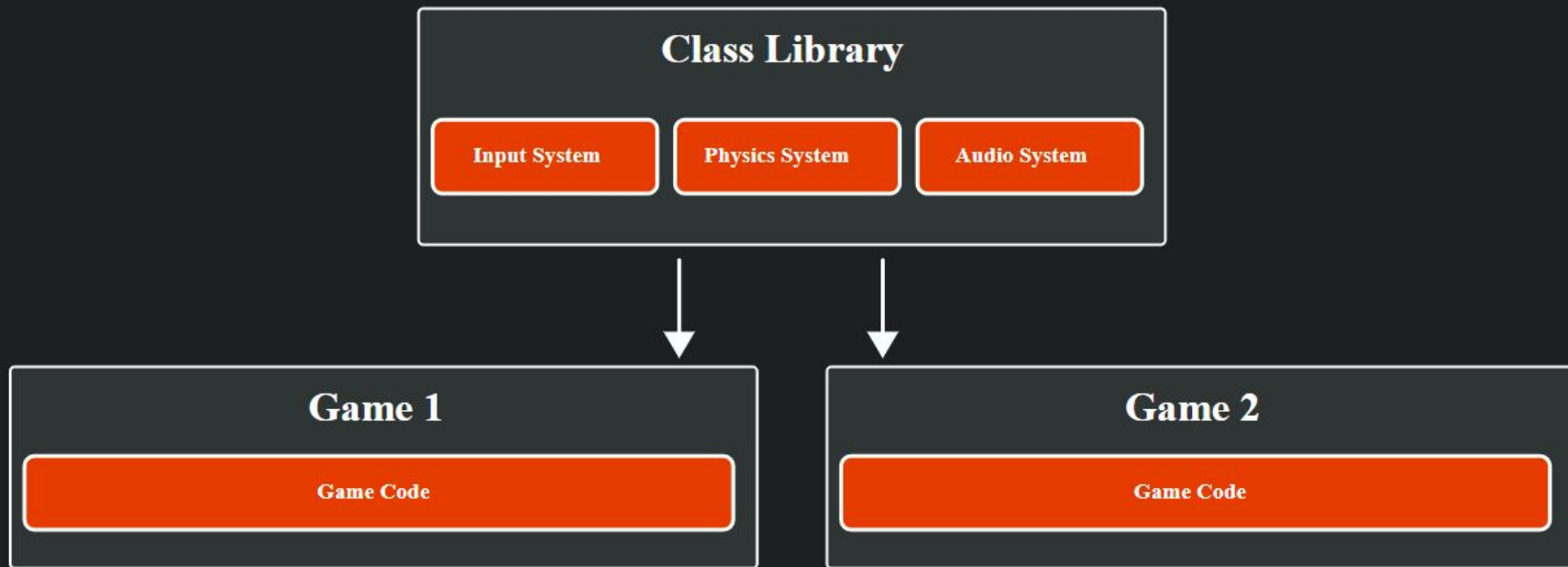
Source Code

- All Source code for today's game can be found here:
 - <https://github.com/Metamate/gmd2-flappy>

Class Library



Class Library



Flappy -1: Class Library

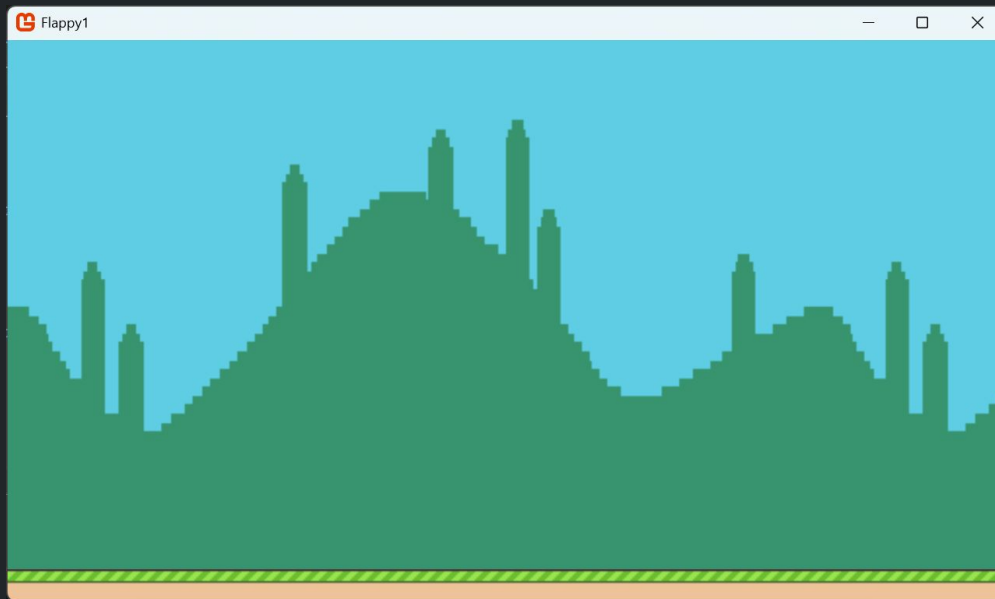
- Create a MonoGame Game Library project “GMDCore”
 - This will be a common core of functionality that we can add to as we create more games
 - It is simply a class library with MonoGame installed as a dependency
- Create a Core class which derives from the Game class
- Add the UpdateScreenScaleMatrix() method from last week to the Core class
 - You will need to declare a Matrix field (consider the visibility), as well as fields to store the virtual width and virtual height (they are no longer constants)
 - Subscribe the method to the Window.ClientSizeChanged event (like last week)
 - Call UpdateScreenScaleMatrix as the game is initialized (like last week)
- Add a constructor to the Core class
 - It must take these arguments
 - string title, int windowWidth, int windowHeight, int virtualWidth, int virtualHeight
 - In the constructor, use the provided arguments
 - Assign values to virtual width and virtual height fields
 - Set the Window.Title property and PreferredBackBufferWidth/Height of the GraphicsDeviceManager
- Make the GraphicsDeviceManager and SpriteBatch visible in derived classes
 - Consider naming conventions when changing visibility of a field
- Move the SpriteBatch initialization to the start of the Initialize() method
- Remove the call to GraphicsDevice.Clear() in the Draw method

Flappy 0: Consuming the Class Library

- Create a new MonoGame project “Flappy”
 - Use the “Cross-Platform Desktop Application”-template (OpenGL)
- Add a reference to the GMDCore class library
 - Derive your Game1 class from the Core class in the class library
 - Take advantage of the Core class constructor to create a window with...
 - 512x288 virtual resolution
 - 1280x720 window resolution
 - The implementation of your Game1 constructor should be empty for now
- Remove the GraphicsDevice & SpriteBatch fields from the Game1 class
 - We are now using the base Core class counterparts

Flappy 1: Background

- Add a [background image](#) and a [ground image](#) to the empty screen
 - Store the two images in an “Images” folder inside your Content folder
 - Remember to call `ContentManager.Load` inside `LoadContent()`



Parallax Scrolling

Illusion of movement given two frames of reference that are moving at different rates



Flappy 2: Parallax

- Implement an infinitely scrolling background using the two provided images
 - The background must move slower than the ground in order to create the desired parallax effect
 - You will need two new constants to control the scroll speed as well as two new fields to keep track of the position of each element
 - For the graphics provided to you, the “looping points” are
 - Background: 413
 - Ground: 512 (virtual width)
 - The % operator might come in handy here

Games are Illusions



Flappy 3: Bird & Assets

- Add a [bird](#) to the screen
 - Create a Bird class to encapsulate the relevant data (for now Position)
 - Remember to add relevant methods (Update and Draw)
- How do we get a reference to the Bird image in the bird class? Multiple approaches...
 - Make the texture available as a public static field in the Game1 class (like last week)
 - This breaks encapsulation and bloats the Game1 class...
 - Inject the Texture2D or ContentManager into the bird upon initialization
 - This is a bit tedious to have to do for every instantiation of an entity
 - Make the ContentManager available across our game files
 - marking it as a static field in the Core library
 - Create a separate, static Art class that holds references to all assets
 - This class would then be passed the ContentManager in its constructor
- Refactor your existing asset references (background and ground) to use the Art class
- Initialize the Bird in the Game1 Initialize method with a default position of 100,100

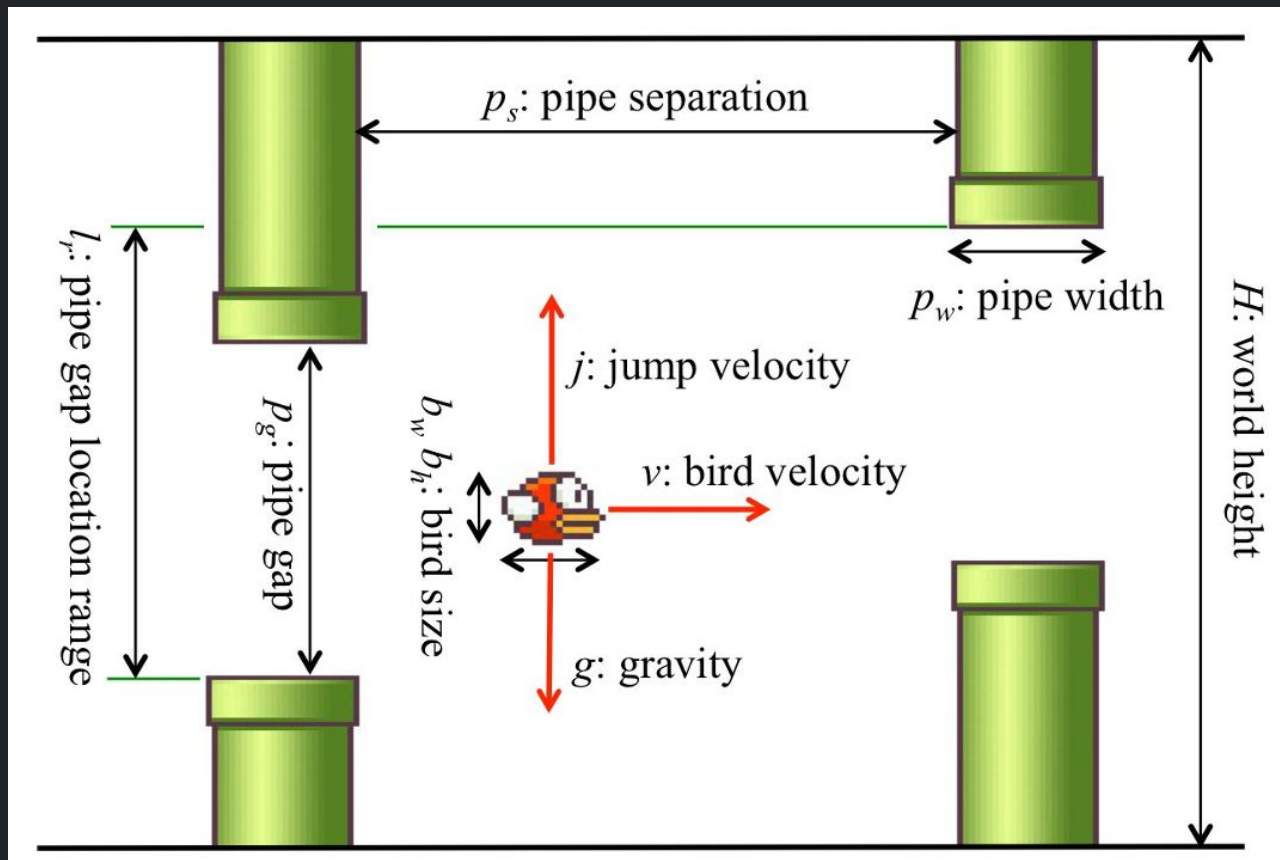
Flappy 4: Gravity

- Add gravity to make the bird fall
 - You will need to introduce two new fields
 - One constant for gravity (pick a fitting value) and another to keep track of the velocity of the bird
 - Remember to normalize both the update in position and velocity by using deltatime

Flappy 5: Flap

- Recall from last week how to emulate a key being pressed down this frame? Add that functionality to the GMDCore library!
 - Create a new folder “Input” in GMDCore
 - Add a class “KeyboardInfo” to the folder containing
 - A way to keep track of previous and current keyboardstate
 - Create a custom Update method to update them each frame
 - Methods for checking IsKeyUp, IsKeyDown, WasKeyJustPressed, WasKeyJustReleased
 - Add another class “InputManager” in the same folder containing
 - A reference to a KeyBoardInfo instance
 - An update method that calls the update method of the KeyBoardInfo class
 - Add the InputManager as static property to the Core class
 - Remember to call Update on the InputManager in the Update method of Core
- Make the bird flap with WasKeyJustPressed(Keys.Space)) to counteract the gravity
 - Remember, the 2D coordinate system has its origin in the top center...
 - Overwriting the vertical velocity with a “jumpforce” value is a good way to instantly give the bird a jolt of upwards momentum

Procedural Generation



Flappy 6: Infinite Pipes

- Add the [pipe](#) to our game
 - Unsurprisingly, we will model the Pipe as a class
 - Inside the class, you must create a constructor that spawns a pipe at a random vertical position at the rightmost edge and within the lower quarter of the screen
 - The Random class will be useful here
 - You will need to add the Pipe as a Texture2D to the Art class
 - You must also create an update method that “scrolls” the Pipe to the left of the screen based on a pipe speed value
 - Picking the same speed as the ground scroll “grounds” the pipes to the ground
 - Include functionality for rendering the Pipe to the screen
- Render the pipe an “infinite” amount of times!
 - In the Game1 class, add functionality to spawn pipes at a set interval
 - You will need two new fields, a spawn timer, and a constant spawn interval
 - You will also need a list to keep track of all pipes
 - You will be iterating through the list of pipes both in Update and Draw of Game1

Dead Zone



Spawn Zone

Flappy 7: Pipe Pairs

- The pipes must now be spawned in “pairs”, with one pipe facing up and the other facing down.
 - We will model this as a new class, PipePair, wrapping the Pipe class
 - The PipePair class will contain
 - A top pipe and a bottom pipe
 - A pipe gap (vertical distance between two pipes)
 - A pipe speed value (move this from the Pipe class to the PipePair class)
 - A float to keep track of the horizontal position of the two pipes
 - A constructor that takes a float y value to define the vertical position of the pipe gap
 - Logic that flags PipePairs for deletion once they go beyond the left side of the screen
 - Since the PipePair will be controlling the pipes, the update and initialization logic of the Pipe class will be moved to the PipePair class
 - The Pipe constructor must now take a position and a bool indicating whether it is flipped
 - The position is purely used to draw the Pipe, as all movement is done in PipePair
 - For flipping the pipe art, you must use a specific [overload of SpriteBatch.Draw\(\)](#) with a [SpriteEffects](#) argument
- Update the Game1 class to work with PipePairs instead of Pipes
 - The pipe pairs will be spawned given a vertical position for the gap
 - Make this position vary slightly for each pipe pair
 - Keep track of the previous spawn position and add a random value (e.g. -50-50) to it
 - This position must be clamped so that you don't risk spawning the pipe gap above/below the screen
 - Add functionality to delete PipePairs flagged for deletion

Collisions

- How can we do collision detection?
 - AABB - same as last week!

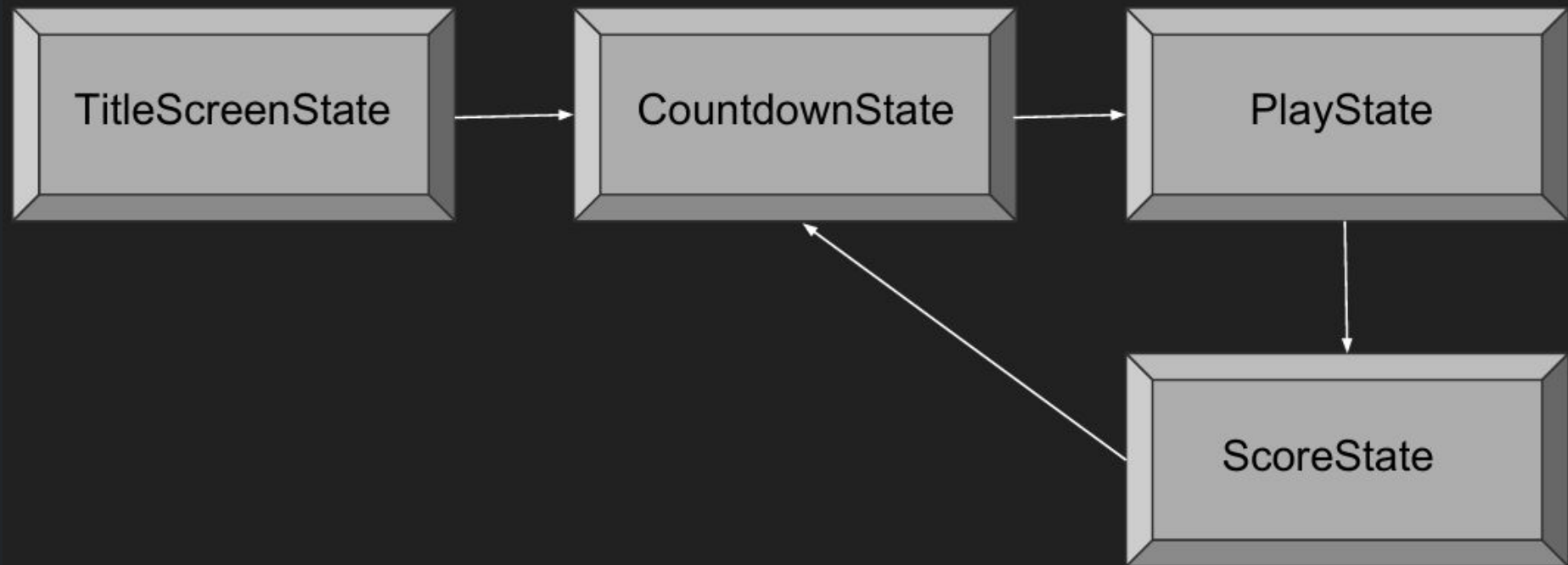
Flappy 8: Collisions

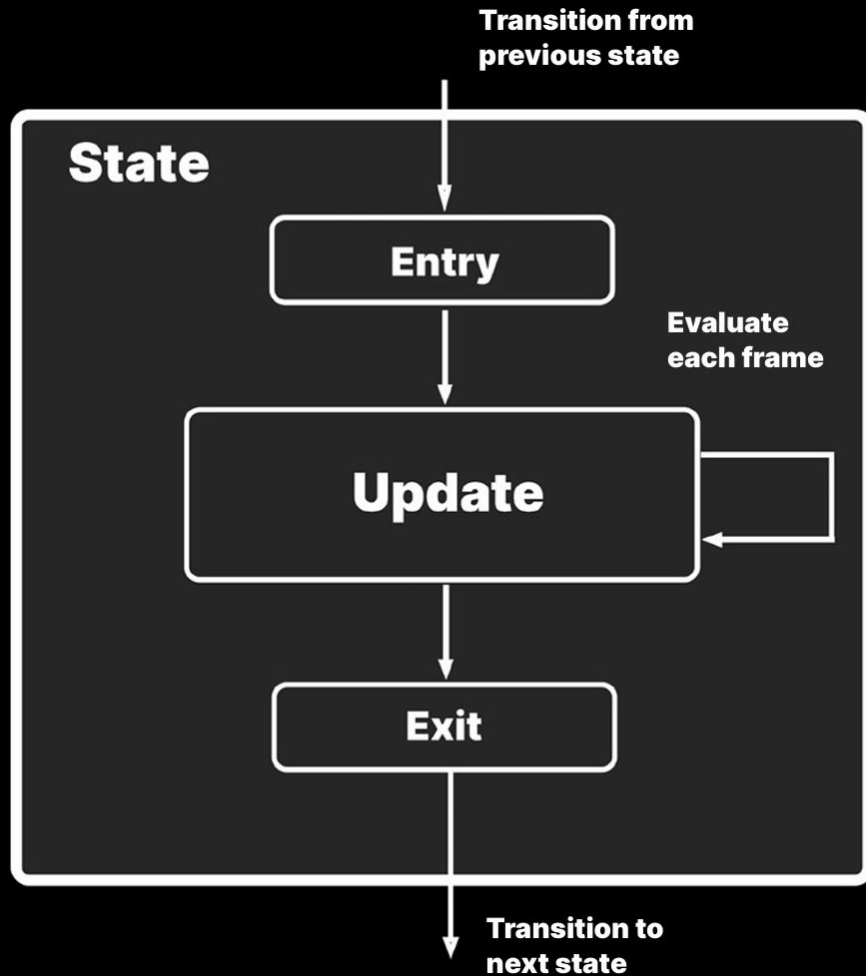
- Include collision detection between bird and pipes
 - When a bird collides with a pipe, the game should pause
 - Pause can be implemented as a simple boolean flag that is checked in Update
 - The Game1 class should react to collisions, but the the actual AABB collision detection should be delegated to the Bird class (similar to last week!)
 - Can you figure out a way to make collision detection more forgiving?
- Add a similar collision check between the bird and ground & ceiling of the game
 - This check can simply be done in Game1 Update
 - It simply compares the bird y position to other values, no AABB collision detection involved...

State

- Do you remember how game state was handled in the Pong game?
 - Why is this potentially “problematic”?

Flappy Bird State Machine





Interfaces

```
public interface IState
{
    void Enter();
    void Exit();
    void Update(GameTime gameTime);
    void Draw(SpriteBatch spriteBatch);
}
```

```
public class PlayState : IState
{
    public void Enter()
    {
    }

    public void Exit()
    {
    }

    public void Update(GameTime gameTime)
    {
    }

    public void Draw(SpriteBatch spriteBatch)
    {
    }
}
```

```
public class StateMachine(Game1 game)
{
    private IState _currentState;

    public TitleState TitleState { get; private set; } = new TitleState(game);
    public PlayState PlayState { get; private set; } = new PlayState(game);

    public void ChangeState(IState newState)
    {
        _currentState?.Exit();
        _currentState = newState;
        _currentState.Enter();
    }

    public void Update(GameTime gameTime)
    {
        _currentState?.Update(gameTime);
    }

    public void Draw(SpriteBatch spriteBatch)
    {
        _currentState?.Draw(spriteBatch);
    }
}
```

Flappy 9: State Machine

- Create a new IState interface
 - The interface must have four methods
 - Enter()
 - Exit()
 - Update(GameTime gameTime)
 - Draw(SpriteBatch spriteBatch)
- Create two new states implementing the interface
 - Title state
 - The user sees a title and presses enter to start playing (i.e. transition to play state)
 - The background should not be scrolling in this state! (repurpose the pause functionality to achieve this)
 - Play state
 - The main state where the game is played (notice how most of your Game1 gameplay code will be moved here)
 - If the bird collides with anything, return to title state
 - This state will need to reset important gameplay values in the Enter method (to ensure a clean state for every retry)
- Create a state machine that controls the global state of the game
 - You need to supply a reference of the Game1 instance to the StateMachine constructor, which in turn makes it available to each state through their constructors.
 - Remember to initialize the state machine and call its Update and Draw methods from the Game1 class
- Add the [retro](#) font to the game
 - There is an overload method of DrawString() that lets you tweak the scale of the drawn text

Flappy 10: Scoring

- Introduce scoring to the game
 - When the bird passes a pipe pair, the score should be incremented by one
 - The score should be tracked in the play state
 - Introduce a property on PipePair to keep track of if that pair has already been scored
 - Display the score in the top left corner of the screen while in play state
- Add a score state that displays the final score
 - Display the score in big text in the center of the screen
 - Consider - how do you get access to the score data from this state?
 - Transition from play state to score state when the bird has collided with an obstacle
 - Transition from score state to play state when space is pressed

Flappy 12: Audio

- Add background music (i.e. a “[Song](#)”) to the game
 - You can use [this track](#)
 - You will need to use the static [MediaPlayer.Play\(Song song\)](#) method to play the song
- Add sound effects to the game
 - You must have three distinct sound effects:
 - [Flap](#)
 - [Hurt](#) (you can play this together with an [Explosion](#) sound for extra impact)
 - [Score](#)
- For organizing the sounds, you can create a new Audio class that works similarly to the Art class, just for audio assets

You Made it (Again)!

Congratulations! You now have your very own working Flappy Bird Game made with MonoGame.



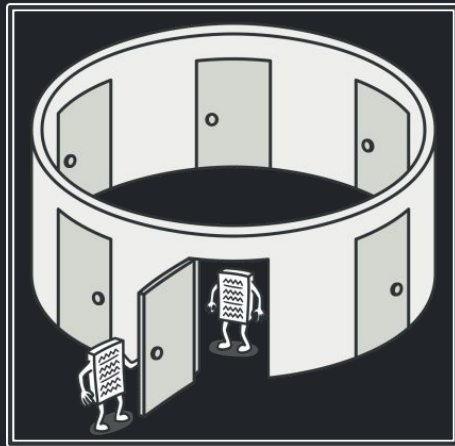
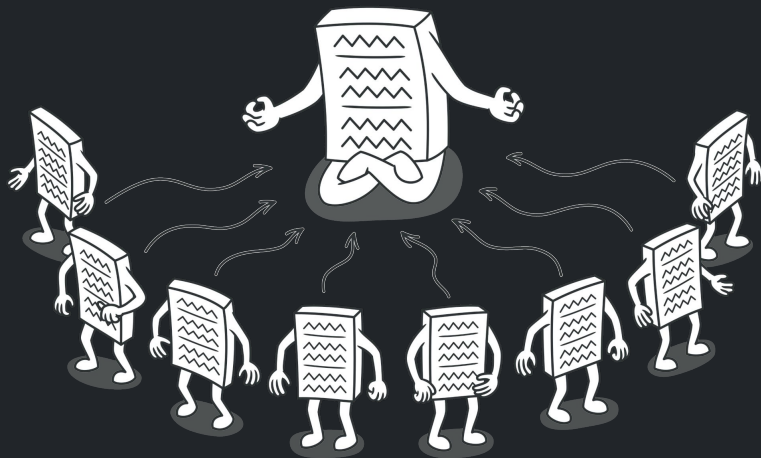
Achievement Unlocked!
Completed the tutorial.

Singleton Pattern: Problem

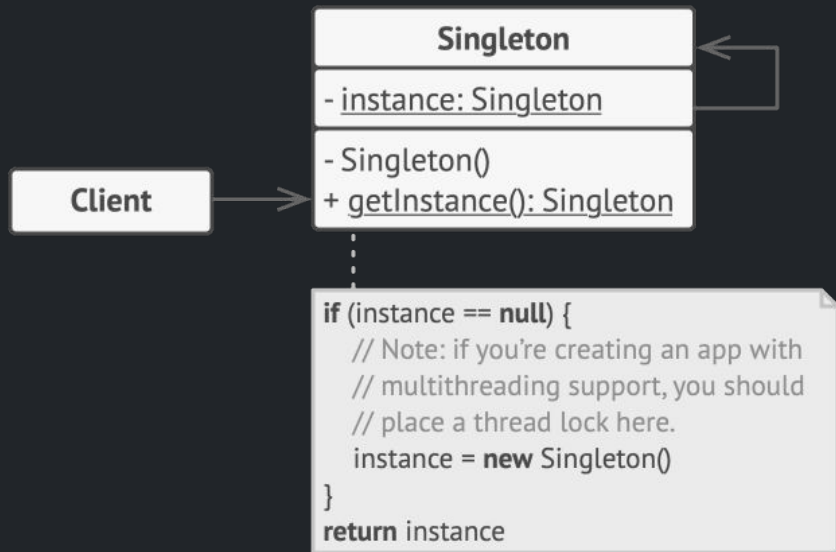
"Ensure that only one instance of the singleton class ever exists

***and** provide global access to that instance."*

- Gang of Four, Design Patterns



Singleton Pattern: Solution



```
public class Singleton
{
    private static Singleton _instance;
    public static Singleton Instance
    {
        get
        {
            _instance ??= new Singleton();
            return _instance;
        }
    }
}
```

